PROYECTO INTERMODULAR 2

APRENDIZAJE DE SPRING BOOT



Miriam Mendieta Racedo.-
2DAMS
10/10/2026

SPRINT 1 – APRENDIZAJE DE SPRINT BOOT

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 10 de Octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---- 1

Promt para la IA --> plan de aprendizaje Sprint Boot ---- 2

Instalación de Spring Boot ---- 4

Bitácora de Aprendizaje (Roadmap) ---- 7

Fase 1 — Fundamentos de Spring e Inversión de Control ---- 10

Fase 2: APIs REST y Capa Web (Spring MVC) ---- 19

Fase 3: Persistencia de Datos y Validación ---- 26

Fase 4: Seguridad y Arquitectura Avanzada ---- 35

Fase 5: Testing, Documentación y Despliegue (Producción) ---- 42

BIBLIOGRAFÍA ---- 52

INTRODUCCIÓN

El ecosistema de ***Spring Boot*** es, sin duda, uno de los pilares más sólidos y populares en el desarrollo backend a nivel mundial.

Su curva de aprendizaje puede resultar abrumadora: anotaciones por todas partes, conceptos que parecen abstractos al principio y una cantidad enorme de magia que ocurre “detrás de escena”.

Precisamente por eso nace esta bitácora: para acompañar, ordenar y dar sentido a mi propio proceso de aprendizaje dentro del framework.

Mi objetivo es construir una documentación clara y progresiva que me permita entender cómo funciona Spring Boot desde sus fundamentos más esenciales hasta las implementaciones avanzadas que hoy exige el mercado laboral.



Promt para la IA → plan de aprendizaje Sprint Boot

Actúa como un Desarrollador Senior en Java y Mentor de Software. Necesito que me expliques Spring Boot desde cero considerando que estoy iniciando un proyecto web con usando Spring Boot 4.x / Java 25, Thymeleaf, Spring Security, MySQL .

Profundiza cada tema, muestra código de ejemplo y manera de llevarlo a mi proyecto, explicando el qué y por qué en cada caso

Explicación conceptual técnica, como si se lo explicaras a alguien que tiene conocimiento básico de java, nada de conexión a base de datos ni framework.

Empezar desde cero paso a paso a explicar spring boot, no tengo conocimientos previos del tema y con java muy básico

Quiero que dividas tu explicación en 3 fases:

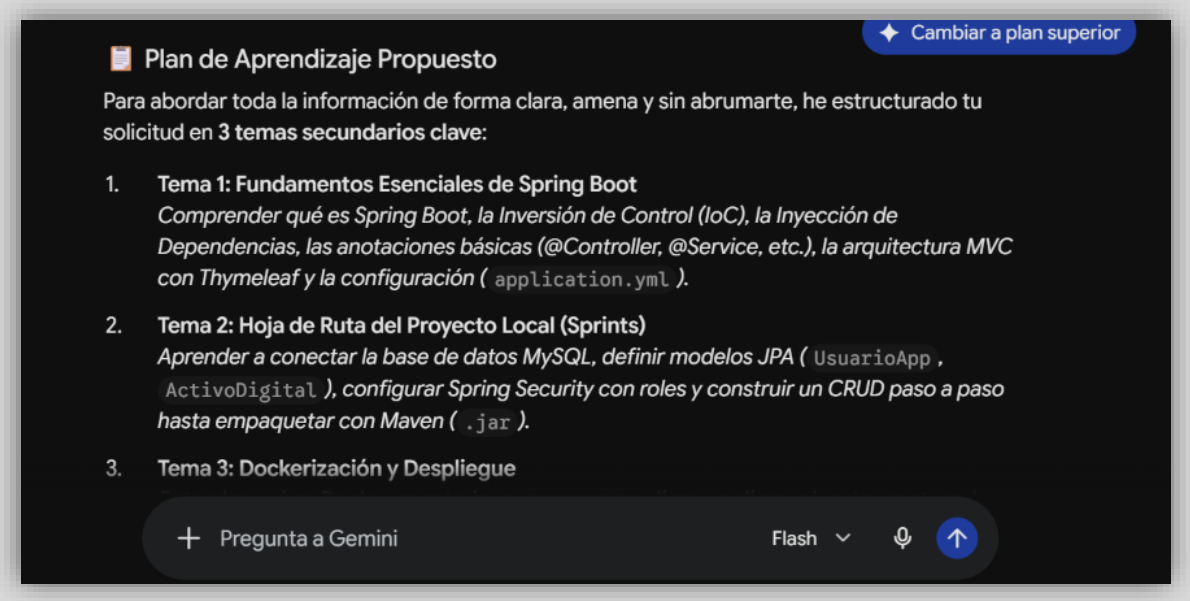
1. FUNDAMENTOS CLAVE DE SPRING BOOT (Explicados con ejemplos sencillos):
    - ¿Qué es spring boot? Paso a paso y desde cero con ejemplos reales para mayor comprensión del tema
    - ¿Qué es Inyección de Dependencias y Control de Inversión (IoC)?
    - ¿Qué hacen las anotaciones principales? (@Controller, @Service, @Repository, @Entity, @Autowired, @Bean).
    - ¿Cómo funciona la arquitectura MVC (Model-View-Controller) conectada con Thymeleaf?
    - ¿Cómo se gestionan las configuraciones en application.properties / application.yml?

2. HOJA DE RUTA PASO A PASO EN LOCAL (Sprint a Sprint):
    - Paso 1: Configurar la conexión a MySQL local y verificar la generación automática de tablas JPA.
    - Paso 2: Crear el modelo de datos básico (UsuarioApp, Rol) y cargar datos iniciales.

- Paso 3: Configurar Spring Security básico (Login con formulario HTML/Thymeleaf y roles RBAC).
- Paso 4: Crear un CRUD completo (Controller -> Service -> Repository -> HTML) para la entidad ActivoDigital.
- Paso 5: Preparar el empaquetado del archivo .jar con Maven (`mvn clean package`).

3. DOCKERIZACIÓN Y DESPLIEGUE FINAL:
   - ¿Qué es Docker y por qué creamos una imagen al final?
   - Estructura y explicación línea por línea del archivo Dockerfile para la app Java.
   - Comandos exactos para construir la imagen (`docker build`) y ejecutar el contenedor (`docker run`).
   - Ejemplo opcional de docker-compose.yml para levantar la app y MySQL juntos en contenedores.

Adopta un tono didáctico, directo, con código de ejemplo claro para mi proyecto y explicaciones breves para no abrumar.



Instalación de Spring Boot

1. Instalación de las herramientas básicas para trabajar con Spring Boot
   - Tener instalado el Java JDK https://www.oracle.com/java/technologies/downloads/
   - Configuración del IDE a utilizar → vs code extensiones:
     - Spring Boot Tools : Nos permite una gestión mas sencilla de las clases Java a la hora de manejarlas con Spring Framework asi como intellisense a la hora de rellenar los ficheros de aplication.properties etc.
     - Spring Initializr: Una extensión que nos permite directamente desde Visual Studio montar un proyecto de Spring boot.
     - Spring DashBoard: Una vista de Visual studio que nos permite gestionar de forma cómoda los proyectos desarrollados con Spring Boot.
     - Spring Boot Extension Pack : Incluye las anteriores y es lo que se recomienda instalar

2. Inicializa el proyecto
   Se puede hacer en dos formas desde el IDE o desde la web de Spring Initializr:

   Método 1: Desde Visual Studio Code

   Si ya instalaste el Spring Boot Extension Pack, puedes hacer todo el proceso anterior sin salir del editor.
   1. abrir Studio Code y en la paleta de comandos presionando Ctrl + Shift + P (en Windows/Linux) o Cmd + Shift + P (en Mac).
      - Escribir Spring Initializr y selecciona la opción Spring Initializr: Create a Maven Project... (o Gradle).
      - Especifica la versión de Spring Boot: Selecciona la versión estable recomendada.
      - Elige el lenguaje: Selecciona Java.
      - Group ID: Escribe el paquete raíz (ej. com.ejemplo) y presiona Enter.
      - Artifact ID: Escribe el nombre del proyecto (ej. mi-api) y presiona Enter.
      - Packaging type: Selecciona Jar.
      - Java version: Elige tu versión de Java (ej. 25).

- Dependencies: Ve buscando y seleccionando las dependencias (ej. escribe Spring Web, DevTools, haz clic en ella). Enter
- Eligir la carpeta donde guardar el proyecto y clic en "Generate into this folder".

2. Cómo ejecutar el proyecto por primera vez
   Una vez que el proyecto esté abierto en VS Code, dale unos segundos a la extensión de Java para que cargue todas las dependencias.
   Verás que en la barra lateral izquierda aparecerá una pestaña llamada SPRING BOOT DASHBOARD.
   Abre esa pestaña, verás el nombre de tu aplicación, pasa el cursor por encima y haz clic en el icono de Play (Iniciar).
   Alternativa: Busca el archivo principal dentro de src/main/java/.../TuAplicacion.java, ábrelo y haz clic en el botón "Run" que aparece justo arriba de la línea public static void main.
   Abre tu navegador web e ingresa a http://localhost:8080. Si ves una página de error que dice "Whitelabel Error Page", ¡felicidades! Significa que tu servidor Spring Boot ya está corriendo perfectamente.

Método 1: Desde la web de Spring Initializr

1. Entra a la web oficial: start.spring.io.
2. Configura los metadatos del proyecto en el panel:
   - Project: Selecciona Maven (o Gradle, según tu preferencia).
   - Language: Selecciona Java.
   - Spring Boot: Deja la versión que viene seleccionada por defecto (evita las que dicen *SNAPSHOT* o *M*, ya que son versiones de prueba).
   - Project Metadata: Define el Group (ej. com.miempresa) y el Artifact (el nombre de tu app, ej. tienda).
   - Packaging: Selecciona Jar.
   - Java: Elige la versión que tengas instalada en tu equipo (ej. 25)
   - Añade las dependencias en el botón "Add Dependencies" del panel derecho. Para empezar, te recomiendo buscar e incluir:
     Spring Web: Indispensable para crear aplicaciones web o APIs REST.
     Spring Boot DevTools : Reinicia la aplicación automáticamente cada vez que haces cambios en el código.

- Haz clic en el botón inferior "Generate". Se descargará un archivo .zip.
- Crear carpeta para descomprimir el .zip

3. Abre el proyecto en VS Code: Descomprime el archivo .zip, abre Visual Studio Code, ve a *File > Open Folder...* y selecciona la carpeta que acabas de descomprimir.



Bitácora de Aprendizaje (Roadmap)

Fase 1: Los Fundamentos de Spring (La Base)

Antes de escribir código web, debes entender el "motor" del framework.

- **Java Esencial:** Domina la Programación Orientada a Objetos, Colecciones (List, Map), Programación Funcional (Streams y Lambdas) y el manejo de excepciones.
- **Inversión de Control (IoC) y Contenedor:** Aprende qué es el **Application Context** y cómo Spring gestiona los objetos de tu aplicación.
- **Inyección de Dependencias (DI):** Entiende la diferencia entre inyectar con la anotación @Autowired por campo, por setter y **por constructor** (la práctica recomendada).
- **Ciclo de Vida de los Beans:** Qué es un Bean, sus alcances (@Scope como Singleton y Prototype) y anotaciones de estereotipo (@Component, @Configuration).

Fase 2: APIs REST y Capa Web (Spring MVC)

Aquí aprendes a comunicar tu aplicación con el mundo exterior.

- **Protocolo HTTP:** Domina los métodos (GET, POST, PUT, DELETE) y los **Códigos de Estado HTTP** (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Error).
- **Controladores REST:** Uso de las anotaciones @RestController, @RequestMapping, @GetMapping y @PostMapping.
- **Parámetros de Entrada:** Cómo recibir datos usando @PathVariable, @RequestParam y mapear el cuerpo de peticiones JSON con @RequestBody.
- **Manejo Global de Errores:** Implementar una clase centralizada con las anotaciones @RestControllerAdvice y @ExceptionHandler.

Fase 3: Persistencia de Datos y Validación

Aprende a conectar tu aplicación a una base de datos relacional y a proteger la integridad de la información.

- **Spring Boot Starters:** Configuración del archivo application.properties o application.yml para enlazar bases de datos (H2 para pruebas, PostgreSQL o MySQL para desarrollo).
- **Mapeo del Modelo (JPA / Hibernate):** Uso de anotaciones esenciales como @Entity, @Table, @Id y @GeneratedValue.
- **Spring Data JPA:** Creación de interfaces que heredan de JpaRepository o CrudRepository y uso de **Derived Queries** (métodos automáticos como findByNombre).
- **Arquitectura DTO y Validación:** Separar las entidades de la base de datos de los datos de entrada usando **DTOs** e implementar validaciones automáticas con **@Valid** y restricciones (@NotBlank, @NotNull, @Size).

Fase 4: Seguridad y Arquitectura Avanzada

El punto de inflexión que separa a un desarrollador Junior de un **Mid/Senior**.

- **Spring Security:** Comprender la **Cadena de Filtros (FilterChain)** y cómo interceptar peticiones HTTP.
- **Autenticación y Autorización:** Configurar accesos basados en Roles y Permisos (.hasRole("ADMIN")).
- **Cifrado de Contraseñas:** Implementar **BCryptPasswordEncoder** para almacenar hashes seguros en la base de datos.
- **Seguridad Stateless con JWT:** Diseñar un flujo donde el cliente se autentica, recibe un **JSON Web Token**, y lo envía en la cabecera Authorization: Bearer [token] en cada petición.

Fase 5: Testing, Documentación y Despliegue (Producción)

Preparar tu aplicación para entornos reales de trabajo.

- **Pruebas Unitarias (Testing):** Escribir tests para la lógica de negocio en la capa de servicios usando **JUnit 5** y **Mockito** para simular (@Mock) el comportamiento del repositorio.
- **Pruebas de Integración:** Evaluar los controladores simulando peticiones HTTP reales mediante el uso de **MockMvc**.
- **Documentación con Swagger / OpenAPI:** Configurar librerías como springdoc-openapi para generar una interfaz interactiva donde se puedan probar los endpoints automáticamente.
- **Spring Boot Actuator:** Monitorear la salud de la aplicación (métricas, estado de la base de datos) en producción.
- **Dockerización:** Crear un Dockerfile para empaquetar tu aplicación en un contenedor listo para desplegar en la nube (AWS, Render, Railway).

Consejos:

1. Empieza una API de una tienda, biblioteca o catálogo. En la Fase 2 crea los endpoints, en la Fase 3 conéctale una base de datos, en la Fase 4 protégela con JWT y en la Fase 5 documéntala y pruébala.
2. **No uses Spring Initializr a ciegas:** Cada vez que agregues una dependencia en start.spring.io, investiga qué librerías exactas añade al proyecto.

Fase 1 — Fundamentos de Spring e Inversión de Control

1. **Java Esencial: La base sobre la que Spring se sostiene**
   El aprendizaje comienza reforzando los pilares del lenguaje Java, porque Spring Boot no es un framework aislado: extiende y automatiza patrones que ya existen en Java, es por esto que dediqué un tiempo a repasar conceptos .

   - **Programación Orientada a Objetos (POO)** Comprender clases, objetos, herencia, interfaces y polimorfismo permite entender cómo Spring organiza y gestiona los componentes de la aplicación. Spring trabaja con objetos que tú defines, pero los administra de forma automática.
   - **Colecciones (List, Set, Map)** Son esenciales para manejar datos dentro de servicios, controladores y repositorios. Ejemplo: una lista de usuarios, un mapa de configuraciones, etc.
   - **Programación Funcional (Streams y Lambdas)** Spring Boot se integra muy bien con operaciones funcionales para transformar datos, filtrar colecciones y simplificar lógica.
   - **Manejo de Excepciones** Fundamental para controlar errores en servicios, validaciones y controladores REST. Spring ofrece mecanismos como @ControllerAdvice y @ExceptionHandler, pero primero debes dominar las excepciones en Java.

2. **Inversión de Control (IoC)**
   En Spring Boot, hacer un new de un componente es un error arquitectónico. Aquí es donde entran los dos conceptos más importantes del framework: la Inversión de Control (IoC) y la Inyección de Dependencias (DI).

   La Inversión de Control es el principio que define a Spring. En lugar de que tú crees y administres los objetos, **Spring se encarga de construirlos, configurarlos y entregártelos cuando los necesitas**. Esto permite aplicaciones más limpias, modulares y fáciles de probar.

El **Application Context** es el contenedor central de Spring IoC. Imagínalo como una caja fuerte en la memoria RAM de tu aplicación que se crea en el mismo milisegundo en que ejecutas el método main().

¿Qué hace este contenedor?

1. **Instancia:** Lee elcódigo, busca clases y crea los objetos por ti.
2. **Configura:** Les asigna las propiedades necesarias (como credenciales de bases de datos).
3. **Gestiona el Ciclo de Vida:** Controla cuándo nace el objeto y cuándo se destruye (cuando se apaga el servidor).

Cualquier objeto que viva dentro de este contenedor deja de ser un objeto común de Java y pasa a llamarse **Bean**.

Para que Spring sepa qué clases debe gestionar y transformar en Beans, utilizamos las **Anotaciones de Estereotipo**. Al colocar estas etiquetas sobre una clase, le estamos diciendo al Application Context: *"Crea un objeto de esta clase y guárdalo en tu contenedor"*.

Las etiquetas esenciales:

- **@Component:** Es la anotación genérica. Se usa para cualquier clase de utilidad general (ej. un encriptador de texto, un procesador de archivos).
- **@Service:** Le dice a Spring que esa clase guarda la lógica de negocio, las reglas y los cálculos de tu aplicación.
- **@Repository:** Indica que la clase maneja el acceso a datos (habla con la base de datos). Además, tiene la propiedad de traducir los errores de la BD a excepciones legibles de Spring.
- **@RestController:** Indica que la clase recibirá peticiones web HTTP y transformará las respuestas a formato JSON de manera automática.

La **Inyección de Dependencias (DI)** y la etiqueta **@Autowired**es es la forma en que Spring proporciona los objetos que ya administra en el contenedor.
Una vez que el Application Context ha escaneado el proyecto y tiene todos los Beans guardados en su memoria, necesita **conectarlos** entre sí. A este proceso de "cableado" automático se le conoce como Inyección de Dependencias.

Formas de inyección:

1. **Inyección por Atributo (No recomendada)**

```
@Service
public class ProductoService {
@Autowired
        private ProductoRepository repository; // Spring la inyecta directamente aquí
}
```

   Hace que el código sea difícil de probar en testing unitario (JUnit) porque no puedes simular (mockear) el repositorio fácilmente sin levantar todo el framework de Spring.

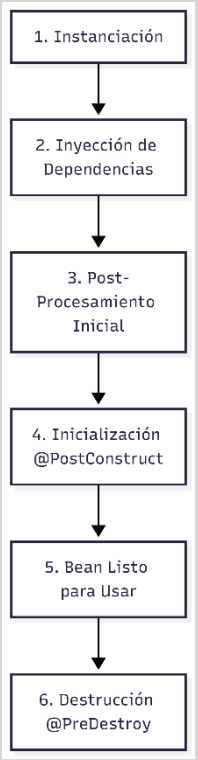
2. **Inyección por Constructor (recomendada)**
   A partir de Spring 4.3, si una clase tiene un solo constructor, Spring inyecta las dependencias automáticamente sin necesidad de escribir la palabra @Autowired.

```
@Service
public class ProductoService {
        private final ProductoRepository repository; // Uso de 'final' para garantizar inmutabilidad
        // Spring ve este constructor y busca el Bean ProductoRepository en su contenedor
        public ProductoService(ProductoRepository repository) {
                this.repository = repository;

        }
}
```

* Inmutabilidad**:** Al usar final, aseguras que la dependencia no cambie a mitad de la ejecución.
* Facilidad de Testing: En un test unitario, puedes hacer new ProductoService(miRepositorioMock) de forma directa y limpia, sin trucos.

3. **Ciclo de Vida de los Beans: Cómo Spring crea y gestiona los objetos**
   Un **Bean** es cualquier objeto que Spring crea y administra dentro del contenedor.
   Cuando ejecutas una aplicación de Spring Boot, los Beans pasan por una serie de etapas perfectamente coordinadas por el Application Context, que se divide en las siguientes fases consecutivas:



https://mermaid.ai/app/projects/

1. **Instanciación:** Spring encuentra la clase (por ejemplo, mediante @Service) y ejecuta su constructor usando reflexión (hace el new interno).
2. **Inyección de Dependencias:** Spring busca todas las dependencias requeridas (por constructor o @Autowired) y las "cablea" en el objeto recién creado.
3. **Post-Procesamiento:** Interfaces internas del framework revisan el Bean para aplicar configuraciones avanzadas (como envolverlo en lógica de seguridad o transacciones).
4. **Inicialización (@PostConstruct):** Se ejecuta la lógica de preparación personalizada del Bean.
5. **Bean Listo:** El Bean se queda viviendo felizmente en el Application Context, listo para atender peticiones de la aplicación de manera segura.
6. **Destrucción (@PreDestroy):** Cuando la aplicación se apaga (por ejemplo, al detener el servidor), Spring limpia la memoria y ejecuta la lógica de cierre.

Alcances (@Scope)

- Singleton (por defecto) Un único objeto para toda la aplicación. Ideal para servicios, repositorios y lógica de negocio.
- Prototype → se crea un nuevo objeto cada vez que se solicita. Útil para objetos con estado mutable.

  Estas anotaciones indican a Spring qué debe gestionar como Bean:
  - @Component → Componente genérico.
  - @Service → Lógica de negocio.
  - @Repository → Acceso a datos.
  - @Controller / @RestController → Controladores web.
  - @Configuration → Clases que definen Beans manualmente.

A veces, necesitas ejecutar código justo cuando un Bean se acaba de crear (como cargar datos iniciales en memoria) o justo antes de que muera (como cerrar una conexión abierta de forma segura). Para ello, Spring nos provee dos anotaciones fundamentales:

1. @PostConstruct (El Bloque de Bienvenida)
Se coloca sobre un método dentro de tu Bean. Spring ejecutará este método **automáticamente una sola vez**, exactamente después de que el Bean haya sido instanciado y todas sus dependencias hayan sido inyectadas con éxito.

¿Por qué no usar el constructor común para esto? Porque en el constructor, las dependencias inyectadas (como los repositorios) **aún podrían ser nulas**. En @PostConstruct, Spring te garantiza que todo el "cableado" terminó y puedes usar tus dependencias de forma segura.

2. @PreDestroy (El Bloque de Despedida)
Se ejecuta **una sola vez justo antes** de que el Application Context sea destruido y la aplicación se apague por completo. Es el momento ideal para liberar recursos.

Sección Especial: Autoevaluación

- **Pregunta 1**: ***Si @Service, @Repository y @Controller hacen técnicamente lo mismo por detrás que @Component, ¿por qué no usamos simplemente @Component para todo?***

  Respuesta Técnica:
  Por dos razones principales: **Semántica** y **Tratamiento Especializado**.

  - **Semántica:** Ayuda a los desarrolladores y a herramientas de análisis de código a entender la arquitectura de la aplicación (capa de negocio, datos o web).

- **Tratamiento Especializado:** Spring añade comportamientos extra a las anotaciones específicas. Por ejemplo, @Repository no solo registra el Bean, sino que activa un post-procesador que traduce automáticamente las excepciones nativas de la base de datos (como las de JDBC o Hibernate) en la jerarquía de excepciones de Spring (DataAccessException). @RestController combina @Controller con @ResponseBody para transformar las respuestas a JSON automáticamente.

- **Pregunta 2:** ***¿Qué es un "Bean" en Spring y en qué se diferencia de un objeto común de Java (POJO)?***

  Respuesta Técnica:
  Un objeto común de Java (POJO) es una instancia que tú creas manualmente usando la palabra clave new y cuyo ciclo de vida tú controlas.

  Un **Bean** es un objeto que ha sido instanciado, configurado y gestionado directamente por el contenedor de inversión de control (IoC) de Spring (el **Application Context**). Un Bean vive dentro del contenedor, lo que le permite recibir inyección de dependencias, participar en transacciones automáticas y tener un ciclo de vida controlado por el framework.

- **Pregunta 3:** ***¿Cuál es el "Scope" (Alcance) por defecto de un Bean en Spring Boot y qué implicaciones tiene en entornos multi-hilo (como una API Web)?***

  Respuesta Técnica:
  El Scope por defecto es **Singleton**. Esto significa que Spring creará **una única instancia** de ese Bean para toda la aplicación y la compartirá cada vez que alguien la solicite.

  Implicación en entornos multi-hilo: Las APIs web procesan múltiples peticiones simultáneamente en hilos concurrentes. Como el Bean Singleton es compartido por todos los hilos, **no debe almacenar variables de estado concurrentes (stateful)** en sus atributos. Si un hilo modifica una variable global del servicio, afectará a todos los demás hilos, provocando errores

catastróficos de concurrencia. Los Beans de Spring deben ser **stateless** (sin estado), limitándose a procesar los datos que entran por los parámetros de sus métodos.

- **Pregunta 4:** ***¿Por qué la inyección por constructor es considerada la mejor práctica por encima de @Autowired sobre el atributo?***

  Respuesta Técnica:
  Por tres razones fundamentales:

  1. **Inmutabilidad:** Al usar el constructor, podemos declarar las dependencias como campos privados y final (private final), garantizando que no se puedan modificar ni reasignar una vez creado el Bean.
  2. **Pruebas Unitarias (Testing):** Facilita los tests con JUnit/Mockito. Si usamos inyección por atributo, estamos obligados a levantar el contexto de Spring (lo cual es lento) o usar reflexión para inyectar los Mocks. Con el constructor, simplemente hacemos un new MiServicio(miMock) en código puro Java.
  3. **Evita dependencias nulas ocultas:** Si intentas instanciar la clase manualmente en un test o en otra sección y olvidas una dependencia, el compilador fallará inmediatamente en lugar de lanzar un NullPointerException en tiempo de ejecución.

- **Pregunta 5:** ***¿Qué pasa si tienes dos clases diferentes que implementan la misma interfaz e intentas inyectar esa interfaz usando @Autowired?***

  Respuesta Técnica:
  Spring Boot entrará en conflicto al arrancar y lanzará una excepción de tipo NoUniqueBeanDefinitionException porque no sabrá cuál de las dos implementaciones elegir para la inyección.

Cómo solucionarlo:

- Opción A: Usar la anotación **@Primary** en una de las implementaciones para indicarle a Spring que esa es la opción prioritaria por defecto.
- Opción B: Usar la anotación **@Qualifier("nombreDelBean")** en el punto de inyección para especificar exactamente el nombre de la clase que queremos utilizar.

- **Pregunta 6: ¿Para qué sirve @PostConstruct?**

Respuesta Técnica:
Sirve para inicializar configuraciones o ejecutar lógica que dependa de objetos externos, garantizando que todas las dependencias de la clase ya han sido inyectadas correctamente por Spring, evitando así errores de tipo NullPointerException.

Fase 2: APIs REST y Capa Web (Spring MVC)

El módulo Spring MVC (Model-View-Controller) es el encargado de interceptar, procesar y responder a las peticiones web en Spring Boot. En el desarrollo moderno de microservicios y SPAs (Single Page Applications), nos enfocamos exclusivamente en la creación de APIs REST que intercambian datos puros en formato JSON, delegando la capa visual por completo al cliente (Frontend/Móvil).

El objetivo es comprender cómo Spring MVC permite construir APIs REST que exponen funcionalidades de la aplicación al exterior mediante el protocolo HTTP. Esta capa web es fundamental para desarrollar servicios backend modernos, interoperables y escalables.

1. ***El Protocolo HTTP y los Códigos de Estado***
   Para diseñar APIs profesionales, debemos respetar la semántica del protocolo HTTP. No usamos las rutas al azar; cada acción del cliente tiene un método específico y cada respuesta del servidor tiene un código numérico estándar.

   Los Métodos HTTP (Verbos)
   Cada método representa una operación estándar sobre recursos:
   - GET → Recuperación de información.
   - POST → Creación de nuevos recursos.
   - PUT→ Actualización completa de un recurso existente.
   - DELETE → Eliminación de un recurso.

   Estos métodos permiten construir una API con una interfaz uniforme y predecible.

   Los Códigos de Estado (HTTP Status Codes)
   Comunican el resultado de la operación:
   - 200 OK → Operación exitosa. Se usa habitualmente en respuestas GET o PUT.
   - 201 Created → Éxito específico para peticiones POST. Confirma que el nuevo recurso se guardó correctamente.
   - 400 Bad Request → Error en los datos enviados por el cliente. Los datos enviados no cumplen con el formato.
   - 404 Not Found → Recurso no encontrado.

- 500 Internal Server Error → Error inesperado en el servidor. Ocurrió una excepción no controlada en nuestro código Java (ej. se cayó la conexión a la base de datos).

El uso adecuado de estos códigos es fundamental para que los clientes interpreten correctamente las respuestas de la API.

2. Controladores REST: **@RestController** vs **@Controller**
   Un controlador REST define los endpoints y la lógica que se ejecuta cuando un cliente realiza una petición.

   Anotaciones principales

   - @RestController Indica que la clase es un controlador REST. Combina @Controller y @ResponseBody, haciendo que los métodos devuelvan datos directamente en formato JSON.
   - @RequestMapping Define la ruta base del controlador o de un método. Permite agrupar endpoints relacionados bajo un mismo prefijo.
   - @GetMapping, @PostMapping, @PutMapping, @DeleteMapping Versiones especializadas de @RequestMapping para cada método HTTP. Facilitan la lectura y organización del código.

     ```
     @RestController
     @RequestMapping("/api/productos")
     public class ProductoController { ... }
     ```

3. ***Parámetros de Entrada: Cómo la API recibe datos***

Un controlador REST puede recibir datos del cliente de tres formas principales. Es una pregunta clásica de arquitectura saber cuándo usar cada una:

1. @PathVariable (Variables de Ruta)

Se utiliza para identificar un recurso específico de forma directa en la URL. Es obligatorio para mantener el estándar RESTful al buscar, actualizar o eliminar registros por su identificador único.

- **URL:** GET http://localhost:8080/api/productos/5

```
@GetMapping("/{id}")
public Producto obtenerPorId(@PathVariable Long id) { ... }
```

2. @RequestParam (Parámetros de Consulta / Query Params)

Se utiliza para datos opcionales, filtros, paginación o búsquedas cruzadas. Van después del signo ? en la URL y se separan con el símbolo &.

- **URL:** GET http://localhost:8080/api/productos?categoria=electronica&orden=precio

```
@GetMapping
public List<Producto> listar(@RequestParam String categoria, @RequestParam String orden) { ... }
```

3. @RequestBody (Cuerpo de la Petición)

Se utiliza exclusivamente en métodos POST y PUT. Mapea el bloque de datos JSON masivo enviado por el cliente directamente a un objeto Java. Jackson deserializa el JSON asociando las llaves del texto con los atributos de la clase Java.

- **Cuerpo HTTP:** {"nombre": "Mouse", "precio": 25.0}

```
@PostMapping
public ResponseEntity<Producto> crear(@RequestBody Producto nuevoProducto) { ... }
```

4. **Manejo Global de Errores: Control centralizado de excepciones**
   En una arquitectura limpia, un controlador nunca debe envolver sus llamadas en bloques try-catch para manejar errores web. Una API profesional debe gestionar errores de forma consistente. Spring permite implementar un sistema centralizado para capturar y responder a excepciones.

   Componentes clave

   - ***@RestControllerAdvice*** Define una clase que intercepta excepciones lanzadas por cualquier controlador REST. Funciona como un interceptor global de excepciones de tipo "Aspecto" (AOP). Escucha activamente a todos los controladores de la aplicación.

```
@RestControllerAdvice
public class ManejadorErroresGlobal {

    // Captura específicamente cuando falla la búsqueda de un ID
    @ExceptionHandler(ProductoNoEncontradoException.class)
    public ResponseEntity<Map<String, Object>> manejarNoEncontrado(ProductoNoEncontradoException ex) {
        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now());
        error.put("error", "Recurso no encontrado");
        error.put("mensaje", ex.getMessage());

        // Retornamos el mapa JSON con el código de estado correcto (404)
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }
}
```

- ***@ExceptionHandler*** Permite especificar cómo debe responder la API ante una excepción concreta. Ejemplos:
  - ✓ Validaciones incorrectas
  - ✓ Recursos no encontrados
  - ✓ Errores inesperados

Ventajas del manejo global

- Respuestas homogéneas en toda la API.
- Mejor experiencia para el cliente.
- Código más limpio en los controladores.
- Facilita la trazabilidad y el diagnóstico de errores.

Sección Especial: Autoevaluación

- **Pregunta 1: ¿Qué es el DispatcherServlet en Spring MVC y cuál es su rol en el flujo de una petición?**

  Respuesta Técnica:
  El **DispatcherServlet** es el componente central de Spring MVC y actúa bajo el patrón de diseño **Front Controller**. Es el encargado de recibir *todas* las peticiones HTTP que llegan al servidor web (Tomcat).

  Su rol no es procesar la lógica, sino coordinar el flujo:

  1. Recibe la petición HTTP pura.
  2. Consulta al HandlerMapping para saber qué controlador y método específico (@RestController) deben manejar esa URL.

3. Envía la petición al controlador seleccionado.
4. Toma el resultado devuelto por el controlador y, mediante los HttpMessageConverters (como Jackson), transforma el objeto Java resultante en un JSON (o renderiza una vista) para enviarlo de vuelta en la respuesta HTTP al cliente.

- **Pregunta 2: ¿Cuál es la diferencia semántica y técnica entre los métodos HTTP PUT y PATCH?**

  Respuesta Técnica:
  Ambos métodos se utilizan para actualizar recursos, pero difieren en el alcance de la modificación:

  - **PUT (Actualización Completa):** Reemplaza el recurso existente en su totalidad con los datos enviados en el cuerpo de la petición. Si el JSON enviado omite algún campo del recurso, ese campo se sobrescribirá como nulo o con su valor por defecto. Además, PUT debe ser **idempotente** (ejecutarlo 10 veces idénticas deja al recurso en el mismo estado).
  - **PATCH (Actualización Parcial):** Aplica modificaciones parciales sobre el recurso existente. El cliente solo envía los campos específicos que desea modificar (por ejemplo, solo el precio de un producto) y los demás atributos de la base de datos se mantienen intactos. No siempre se garantiza su idempotencia dependiendo de cómo se implemente la lógica.

- **Pregunta 3: Si un cliente envía una petición enviando texto plano en lugar de un JSON a un endpoint mapeado con @RequestBody, ¿qué excepción lanza Spring Boot y cómo respondería la API por defecto?**

Respuesta Técnica:
Spring Boot lanzará una excepción de tipo **HttpMediaTypeNotSupportedException** (si la cabecera Content-Type no es application/json) o un **HttpMessageNotReadableException** (si el cuerpo no se pudo parsear/deserializar a la clase Java).

Por defecto, si no tenemos un manejador global, el servidor responderá con un código de estado **HTTP 415 Unsupported Media Type** o un **HTTP 400 Bad Request** acompañado de la clásica página de error interna (White Label Error Page), lo cual expone detalles del framework y es una mala práctica de seguridad. Por eso es vital capturarlas en el @RestControllerAdvice.

- **Pregunta 4: ¿Cuándo se debe elegir @PathVariable frente a @RequestParam según las convenciones de diseño RESTful?**

  Respuesta Técnica:
  La elección depende del rol del dato dentro de la jerarquía de la URL:

  - Use **@PathVariable** cuando el dato sea **esencial e identificativo** para localizar el recurso en la base de datos. Forma parte de la identidad del endpoint (ej: /api/productos/{id}). Sin ese parámetro, la ruta pierde el sentido.
  - Use **@RequestParam** cuando el dato sea **opcional, secundario o dinámico**, diseñado para alterar cómo se presentan los datos pero no la entidad en sí. Es el estándar absoluto para aplicar **filtrado** (ej: ?categoria=libros), **ordenamiento** (ej: ?sort=precio) y **paginación** (ej: ?page=0&size=10).

- **Pregunta 5: ¿Por qué no es seguro ni escalable utilizar colecciones genéricas como Map<String, Object> en los parámetros de entrada con @RequestBody en lugar de clases Java bien definidas?**

  Respuesta Técnica:
  Utilizar un Map rompe los principios de diseño de APIs robustas por tres motivos:

1. **Pérdida de Contrato Técnico:** Al usar un Map, la API acepta cualquier par de clave/valor arbitrario. Se pierde el tipado fuerte y la validación en tiempo de compilación.
2. **Imposibilidad de Validación Automática:** No se pueden aplicar las anotaciones de validación estándar (@NotNull, @NotBlank, @Size) de Jakarta Validation, obligándonos a escribir validaciones manuales con bloques if dentro del controlador.
3. **Documentación Deficiente:** Herramientas automatizadas como Swagger/OpenAPI no podrán generar esquemas ni documentar qué campos requiere el endpoint, afectando gravemente la experiencia de los desarrolladores del Frontend que consumen la API.

Fase 3: Persistencia de Datos y Validación

Esta fase también incluye la validación de datos para garantizar la integridad y consistencia de la información que ingresa a la aplicación. La persistencia de datos es el proceso mediante el cual una aplicación almacena información de forma permanente en una base de datos. En Spring Boot, este proceso se realiza principalmente mediante Spring Data JPA, que utiliza Hibernate como proveedor de JPA.

1. ***Spring Boot Starters y Configuración de Entornos***
   Se simplifica la configuración de bases de datos mediante *starters* y archivos de propiedades.

   Bases de datos comunes

   - ✓ **H2** → Base de datos en memoria, ideal para pruebas rápidas.
   - ✓ **MySQL / PostgreSQL** → Bases de datos relacionales para desarrollo y producción.

   Configuración en application.properties o application.yml

   Se definen parámetros como:

- ✓ URL de conexión
- ✓ usuario y contraseña
- ✓ driver JDBC
- ✓ comportamiento de Hibernate (crear, actualizar o validar el esquema)

Ejemplo típico:

properties
spring.datasource.url=jdbc:mysql://localhost:3306/mi_base
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

Objetivo

Permitir que Spring Boot establezca automáticamente la conexión y gestione el ciclo de vida de la base de datos sin configuraciones manuales extensas.

Gestor de dependencias:

xml

```
<!-- Motor de Persistencia de Spring -->
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

2. ***Mapeo del Modelo (JPA / Hibernate)***

   El mapeo del modelo consiste en transformar clases Java en tablas de la base de datos mediante anotaciones JPA.
   Crear un modelo de datos consistente que Hibernate pueda traducir a tablas y relaciones en la base de datos.

   Anotaciones esenciales

   - **@Entity** Indica que la clase representa una tabla.
   - **@Table**(name = "nombre_tabla") Permite especificar el nombre exacto de la tabla.
   - **@Id** Define la clave primaria.
   - **@GeneratedValue** Indica que el valor del ID será generado automáticamente (por ejemplo, autoincremental).

   Ejemplo:

```
@Entity
@Table(name = "usuarios")
public class Usuario {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String nombre;
       private String email;
}
```

3. **Spring Data JPA: Repositorios y consultas automáticas**

Spring Data JPA elimina la necesidad de escribir código SQL manual para operaciones básicas.

Repositorios

Se crean interfaces que extienden:

- JpaRepository → Incluye operaciones CRUD y paginación.
- CrudRepository → Solo operaciones CRUD básicas.

Ejemplo:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
}
```

Derived Queries

Spring genera consultas automáticamente a partir del nombre del método.

Ejemplos:

- findByNombre(String nombre)
- findByEmail(String email)
- findByEdadGreaterThan(int edad)

Ventajas

- Menos código SQL manual
- Consultas rápidas y expresivas
- Integración directa con el modelo JPA

4. **Arquitectura DTO y Validación**
   Para proteger la integridad de los datos y evitar exponer directamente las entidades, se utiliza una arquitectura basada en **DTOs (Data Transfer Objects)**.
   Utiliza la especificación jakarta.validation.constraints para blindar la entrada de datos.

   DTOs
   Un DTO representa los datos que se reciben o envían a través de la API, sin incluir lógica de negocio ni anotaciones de persistencia.
   Ejemplo:

```
public class UsuarioDTO {
        private String nombre;
        private String email;
}
```

   Validación con @Valid
   La anotación @Valid le ordena a Spring activar el validador integrado antes de parsear los datos. Si el JSON que envía el cliente no pasa el filtro, la petición rebota con un HTTP 400 Bad Request y ni siquiera gasta recursos ejecutando el método.

   Restricciones comunes
   - @NotBlank → El campo no puede estar vacío.
   - @NotNull → El campo no puede ser nulo.
   - @Size(min, max) → Control de longitud.
   - @Email → Formato válido de correo electrónico.

Ejemplo:

```
public class UsuarioDTO {

    @NotBlank
     private String nombre;

     @Email
    @NotBlank
     private String email;
}
```

Uso en controladores

```
@PostMapping("/usuarios")
public ResponseEntity<?> crearUsuario(@Valid @RequestBody UsuarioDTO dto) {
        // lógica de creación
}
```

Objetivo

- Separar la capa de persistencia de la capa de entrada/salida.
- Garantizar que los datos recibidos cumplan reglas de negocio y formato.
- Evitar errores y proteger la integridad de la base de datos.

Sección Especial: Autoevaluación

- **Pregunta 1: ¿Cuál es la diferencia entre extender tu repositorio de CrudRepository o de JpaRepository?**

  Respuesta Técnica:
  La diferencia radica en la jerarquía de herencia y las funcionalidades que expone cada interfaz:

  - **CrudRepository:** Es la interfaz base del ecosistema Spring Data. Ofrece exclusivamente funciones genéricas de operaciones CRUD (crear, leer, actualizar y borrar) devolviendo colecciones de tipo Iterable. Es ideal si buscas una abstracción ligera o si trabajas con bases de datos NoSQL.
  - **JpaRepository:** Extiende directamente de PagingAndSortingRepository y de CrudRepository. Por lo tanto, incluye todo lo anterior y añade métodos específicos para persistencia relacional (JPA), tales como la **paginación** avanzada, el **ordenamiento** de datos, el vaciado del contexto de persistencia en lotes (flush()) y la eliminación masiva de registros en una sola consulta. Devuelve colecciones nativas de tipo List.

- **Pregunta 2: ¿Qué es el problema de la consulta "N+1" en JPA/Hibernate y cómo se puede solucionar en Spring Data JPA?**

  Respuesta Técnica:
  El problema N+1 ocurre cuando tienes una entidad con una relación (por ejemplo, un Pedido que tiene muchos Items) configurada con carga perezosa (FetchType.LAZY).

  Si intentas recuperar una lista de *N* pedidos y luego recorres sus ítems, Hibernate ejecutará:

  1. **1 consulta** inicial para traer todos los pedidos.
  2. **N consultas adicionales** (una por cada pedido) para cargar los ítems correspondientes bajo demanda en la base de datos. Esto destruye el rendimiento del sistema de forma catastrófica.

Cómos olucionarlo:
Se soluciona forzando un join en una sola consulta SQL utilizando la anotación **@Query** con un **JOIN FETCH** o aplicando un **@EntityGraph** sobre el método del repositorio para indicarle a Hibernate qué relaciones debe traer de forma inmediata (EAGER) únicamente para esa consulta.

- **Pregunta 3: ¿Cuál es la diferencia entre los valores update y create-drop en la propiedad spring.jpa.hibernate.ddl-auto, y cuál deberías usar en producción?**

  Respuesta Técnica:

  - **update:** Examina el esquema de la base de datos al arrancar y modifica las tablas existentes añadiendo nuevas columnas o tablas según tus clases @Entity. Nunca elimina columnas ni datos existentes.
  - **create-drop:** Elimina todas las tablas existentes, las crea de nuevo desde cero al arrancar la aplicación y las vuelve a eliminar por completo de la base de datos cuando la aplicación se apaga. Se usa solo para entornos de pruebas unitarias.

  En producción: No se debe usar **ninguno** de los dos. En producción la propiedad debe configurarse en **validate** (que solo comprueba si la estructura coincide sin alterar nada) o **none**. Las alteraciones de bases de datos productivas deben delegarse a herramientas profesionales de control de versiones de bases de datos como **Liquibase** o **Flyway**.

- **Pregunta 4: Si una validación de @Valid en un DTO falla, ¿en qué capa de la aplicación se detiene la petición y por qué es eficiente este comportamiento?**

  Respuesta Técnica:
  La petición se detiene en la **capa web (Frontera de la Aplicación)**, justo antes de invocar el método del controlador.Cuando el DispatcherServlet recibe el JSON de la petición HTTP, llama a los validadores de Jakarta antes de mapear los datos a los parámetros del método del @RestController. Si hay fallos, lanza inmediatamente una excepción MethodArgumentNotValidException.

Esto es sumamente eficiente porque el flujo **rebota de forma temprana** (Fail-Fast). Evita que la aplicación gaste recursos abriendo transacciones de base de datos, ejecutando lógica de negocio o procesando memoria con datos que sabemos de antemano que son inválidos o corruptos.

- **Pregunta 5: ¿Qué significa que una entidad JPA se encuentre en estado "Detached" (Disociada) y en qué se diferencia del estado "Managed" (Gestionada)?**

  Respuesta Técnica:

  Depende de si la entidad está vinculada al **Persistence Context** (la sesión activa de Hibernate):

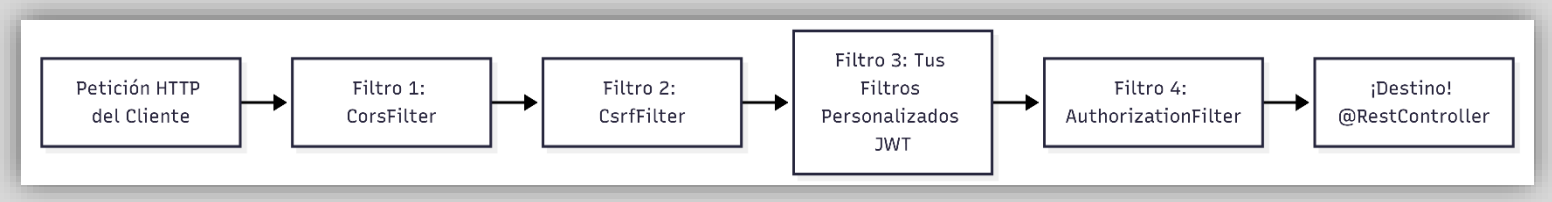
  - **Managed (Gestionada):** La entidad está sincronizada con el contexto de persistencia. Cualquier cambio que hagas en los atributos del objeto Java mediante métodos .set() será detectado automáticamente por Hibernate (mecanismo de *Dirty Checking*) y se guardará de forma automática en la base de datos mediante un UPDATE al finalizar la transacción, sin necesidad de llamar explícitamente al método .save().
  - **Detached (Disociada):** El objeto tiene un ID real en la base de datos, pero ya no está conectado al contexto de persistencia (por ejemplo, porque la transacción se cerró o el objeto se envió serializado a través de la red como respuesta). Si modificas sus atributos, Hibernate ignorará los cambios y no impactará en la base de datos a menos que la vuelvas a vincular usando el método .merge().

Fase 4: Seguridad y Arquitectura Avanzada

La seguridad es uno de los pilares fundamentales en el desarrollo backend. En Spring Boot, la implementación de seguridad se realiza principalmente mediante Spring Security, un framework altamente configurable que permite controlar el acceso a recursos, gestionar autenticación y proteger la aplicación frente a ataques comunes.

Además, esta fase introduce arquitecturas avanzadas como la autenticación stateless con JWT, ampliamente utilizada en APIs modernas.

1. **Spring Security y la Cadena de Filtros (FilterChain)**
   Spring Security funciona mediante una cadena de filtros que intercepta todas las peticiones HTTP antes de que lleguen a los controladores. Cada filtro cumple una función específica, como verificar credenciales, validar tokens o aplicar reglas de autorización. Cuando una petición HTTP viaja desde el cliente hacia tu aplicación, debe atravesar un túnel de inspección obligatorio conocido como la **SecurityFilterChain**.



- Cada filtro tiene una única responsabilidad (verificar tokens, validar cookies contra ataques CSRF, comprobar permisos de rutas, etc.).
- Si un filtro detecta que la petición no cumple con los requisitos de seguridad (por ejemplo, un token expirado o un formato inválido), **rompe la cadena de inmediato** y lanza una excepción web, impidiendo que la petición gaste memoria llegando al controlador.

2. **Autenticación vs. Autorización: Roles y Permisos**
   Spring Security distingue entre dos conceptos fundamentales:

   ✓ Autenticación → Proceso mediante el cual se verifica la identidad del usuario. Ejemplo: usuario + contraseña.

   ✓ Autorización → Proceso que determina qué acciones puede realizar un usuario autenticado. En Spring Security, las rutas se protegen mediante una clase de configuración donde exponemos un Bean de tipo SecurityFilterChain:

```
.authorizeHttpRequests(auth -> auth
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
.anyRequest().authenticated()
)
```

Spring añade automáticamente el prefijo ROLE_ por debajo cuando usas .hasRole("ADMIN"). Por ende, el usuario en la base de datos debe tener asignado el texto exacto ROLE_ADMIN. Si prefieres validar la cadena pura sin prefijos, se utiliza .hasAuthority("ADMIN").

Buenas prácticas

- Usar roles para permisos generales (ADMIN, USER).
- Usar authorities para permisos específicos (READ_PRIVILEGE, WRITE_PRIVILEGE).
- Mantener la lógica de autorización centralizada en la configuración de seguridad.

3. **Cifrado de Contraseñas con BCryptPasswordEncoder**
   Nunca se deben almacenar contraseñas en texto plano, ni aplicar algoritmos reversibles o hashes simples como MD5 o SHA-256 (ya que son vulnerables a ataques por tablas de arcoíris). Spring Security proporciona BCryptPasswordEncoder, un algoritmo de hashing seguro que incluye salting y múltiples rondas de procesamiento.

   Características de BCrypt
   - Genera hashes únicos incluso para contraseñas iguales.
   - Resistente a ataques de fuerza bruta.
   - Recomendado por OWASP.
   - Es unidireccional: Es matemáticamente imposible revertir el hash final para descubrir la contraseña original.
   - Salt Aleatorio Automático: Cada vez que encriptas la misma contraseña (ej: "password123"), BCrypt le añade un valor aleatorio único (sal) antes de procesarla. Esto significa que el texto hash resultante siempre será diferente, impidiendo que un atacante deduzca contraseñas idénticas comparando la base de datos.
   - Costo Adaptable (Work Factor): Permite configurar la potencia del algoritmo para que requiera más tiempo de procesamiento, volviéndolo inmune a ataques de fuerza bruta modernos.

```
@Bean
public PasswordEncoder passwordEncoder() {
   return new BCryptPasswordEncoder();
}
Al registrar un usuario:
usuario.setPassword(passwordEncoder.encode(dto.getPassword()));
```

4. **Seguridad Stateless con JWT (JSON Web Tokens)**
   Las aplicaciones web tradicionales guardan el estado del usuario en la memoria del servidor usando **Sesiones HTTP (JSESSIONID)**. En arquitecturas de microservicios o APIs modernas, esto destruye la escalabilidad, ya que si tienes 5 servidores balanceados, tendrías que sincronizar las sesiones entre ellos.
   La solución es una arquitectura **Stateless (sin estado)** utilizando **JWT**. El servidor no guarda memoria de quién está conectado; la responsabilidad de la identidad se traslada al cliente a través de un token criptográfico.

   El flujo del token paso a paso:

   1. **El Cliente hace Login:** Envía sus credenciales en un JSON al endpoint libre /api/auth/login.
   2. **El Servidor firma el JWT:** El servidor valida que las credenciales coincidan usando .matches(). Si son correctas, empaqueta los datos esenciales del usuario (Username, Roles, Expiración) y los firma criptográficamente usando una **Clave Secreta** guardada en el servidor.
   3. **El Servidor responde:** Entrega el string del token firmado al cliente.
   4. **Consumo Protegido:** Para las siguientes peticiones (ej. crear un producto), el cliente adjunta obligatoriamente el token en la cabecera HTTP:
      - Authorization: Bearer eyJhbGciOiJIUzI1...
   5. **El Filtro Interceptor Valida:** Nuestro filtro personalizado (JwtAuthenticationFilter) intercepta la cabecera, verifica que la firma criptográfica sea auténtica (usando la clave secreta) y, si el token no ha expirado, inyecta la identidad en el contexto de seguridad de Spring de forma instantánea para esa petición en particular.



Sección Especial: Autoevaluación

- **Pregunta 1: ¿Qué es el SecurityContextHolder y qué relación tiene con el ciclo de vida de una petición HTTP en un entorno Stateless?**

  Respuesta Técnica:
  El SecurityContextHolder es el lugar centralizado donde Spring Security almacena los detalles de la identidad del usuario actualmente autenticado (el objeto Authentication).
  Por defecto, funciona utilizando un mecanismo de ThreadLocal. Esto significa que la información de seguridad está vinculada estrictamente al hilo de ejecución actual que está procesando la petición HTTP.

  En un entorno Stateless (sin estado):

  1. Al inicio de la petición, el SecurityContextHolder está vacío.
  2. Nuestro filtro JWT valida el token e inyecta al usuario dentro de este contexto.
  3. El controlador lee la identidad desde este objeto si lo necesita.
  4. Al finalizar la petición HTTP, la cadena de filtros de Spring Security limpia automáticamente el hilo (ThreadLocal.remove()). Así, el hilo regresa limpio al pool del servidor Tomcat y queda listo para atender una petición completamente diferente de otro usuario de forma segura.

- **Pregunta 2: ¿Por qué deshabilitamos la protección CSRF (.csrf(csrf -> csrf.disable())) en una API REST que utiliza autenticación basada en tokens JWT?**

  Respuesta Técnica:
  La vulnerabilidad CSRF (Cross-Site Request Forgery) ocurre principalmente porque los navegadores web adjuntan automáticamente las Cookies en cada petición saliente hacia un dominio específico, sin importar qué sitio web haya originado la

llamada. Un atacante puede aprovechar esto para hacer que el navegador del usuario envíe una petición maliciosa con la sesión ya iniciada.
Deshabilitamos CSRF en APIs REST basadas en JWT porque estas son Stateless y no utilizan cookies ni sesiones en el servidor para rastrear la identidad. En su lugar, el cliente debe incluir manualmente el token en la cabecera Authorization: Bearer [token]. Dado que un sitio web malicioso de terceros no tiene acceso por código JavaScript a las cabeceras personalizadas ni al almacenamiento local (localStorage) del dominio de nuestra API, el ataque CSRF es técnicamente imposible de ejecutar.

- **Pregunta 3: Si dos usuarios diferentes eligen exactamente la misma contraseña (ej: password123), ¿por qué sus hashes en la base de datos se ven completamente distintos al usar BCrypt?**

  Respuesta Técnica:
  Esto ocurre gracias al mecanismo de Salt (Sal) aleatoria integrado de forma nativa en el algoritmo de BCrypt.
  Cada vez que invocamos el método passwordEncoder.encode(), el algoritmo genera de manera interna y automática una cadena aleatoria única de caracteres (la sal) y la concatena a la contraseña en texto plano antes de realizar las rondas de hashing criptográfico. La sal resultante se empaqueta dentro del propio string del hash final almacenado. Como las sales generadas siempre son diferentes para cada invocación, los hashes finales serán completamente distintos, lo que impide a los atacantes usar ataques de fuerza bruta basados en tablas precalculadas (Rainbow Tables / Tablas de Arcoíris).

- **Pregunta 4: ¿Qué es un ataque de "Token Hijacking" (Secuestro de Token) en JWT y qué estrategias arquitectónicas puedes implementar en Spring Boot para mitigar su impacto?**

  Respuesta Técnica:
  El secuestro de token ocurre cuando un atacante logra robar el JWT de un usuario legítimo (por ejemplo, mediante un ataque XSS en el frontend o interceptando el tráfico de red) y lo utiliza para suplantar su identidad en la API. Dado que la API es Stateless, tratará al portador del token como el usuario real.
  Para mitigar este impacto desde el Backend en Spring Boot, implementamos las siguientes estrategias:

1. Tiempos de Expiración Cortos: Configurar el withExpiresAt() del JWT para que venza en plazos cortos (ej. de 15 a 30 minutos).
2. Arquitectura de Refresh Tokens: Emitir un token de acceso de corta duración y un Refresh Token de larga duración almacenado de forma segura (ej. en una cookie HttpOnly y Secure) para renovar el token de acceso sin pedir credenciales constantemente.
3. Firma Criptográfica Asimétrica: Utilizar algoritmos basados en claves públicas/privadas (como RS256 en lugar de HMAC256), de modo que solo el servidor de autorización pueda firmar los tokens usando la clave privada, pero cualquier microservicio pueda validarlos usando la clave pública.

- **Pregunta 5: ¿Por qué es una mala práctica capturar las excepciones de seguridad (como tokens inválidos o expirados) dentro de un @RestControllerAdvice?**

Respuesta Técnica:
Es un error conceptual porque la anotación @RestControllerAdvice forma parte del módulo Spring MVC, el cual opera exclusivamente dentro de la capa del controlador (cuando la petición ya entró al flujo de ejecución de Java Web).
Los filtros de Spring Security actúan mucho antes en el ciclo de vida, en la capa de Filtros Servlet (Tomcat), antes de que Spring MVC tome el control de la petición. Si una excepción de seguridad (como un token manipulado) ocurre dentro de la SecurityFilterChain, el @RestControllerAdvice nunca se enterará de su existencia.
La solución correcta: Las excepciones de filtros de seguridad deben manejarse implementando un componente personalizado que extienda de AuthenticationEntryPoint (para errores 401 de autenticación) o AccessDeniedHandler (para errores 403 de autorización) y registrarlo directamente dentro del Bean de la configuración de seguridad.

Fase 5: Testing, Documentación y Despliegue (Producción)

Escribir código que funcione localmente es solo el primer paso. El software profesional requiere pruebas automatizadas para evitar regresiones (errores al añadir código nuevo), documentación viva para los equipos de Frontend y empaquetamiento aislado para su distribución en servidores en la nube. Estos procesos son esenciales para garantizar calidad, estabilidad y escalabilidad en producción.

1. **Pruebas Unitarias (JUnit 5 + Mockito)**
   El objetivo de una **prueba unitaria** es aislar por completo una clase y probar su lógica matemática o de negocio sin levantar el framework de Spring y sin conectarse a una base de datos real (lo cual haría los tests lentos y dependientes de datos externos). Para lograr este aislamiento en la capa de servicios, se usa **Mockito** para simular (crear "falsificaciones" o *Mocks*) el comportamiento de la capa de datos.

```
@ExtendWith(MockitoExtension.class)
 class UsuarioServiceTest {

        @Mock
        private UsuarioRepository usuarioRepository;
         @InjectMocks
        private UsuarioService usuarioService;

        @Test
         void crearUsuario_deberiaGuardarCorrectamente() {
                Usuario usuario = new Usuario("Miriam", "miriam@mail.com");
                when(usuarioRepository.save(any())).thenReturn(usuario);
                Usuario resultado = usuarioService.crearUsuario(usuario);
                assertEquals("Miriam", resultado.getNombre());
         }
    }
```

2. **Pruebas de Integración (MockMvc)**
   A diferencia de las unitarias, las **pruebas de integración** verifican que múltiples capas interactúen correctamente entre sí. En la capa web, usamos **MockMvc** para simular peticiones HTTP reales (como llamadas de red de Postman) dirigidas a nuestros controladores, evaluando si las rutas, filtros de seguridad, validaciones y códigos de estado responden según lo esperado.

```
@Autowired
private MockMvc mockMvc;

@Test

void deberiaRetornar200EnGetUsuarios() throws Exception {

        mockMvc.perform(get("/usuarios"))

        .andExpect(status().isOk());

}
```

3. **Documentación Viva con OpenAPI / Swagger**
   Una API REST debe estar bien documentada para que los desarrolladores de Frontend o aplicaciones móviles sepan qué endpoints consumir sin necesidad de leer el código fuente Java.
   Para automatizar esto de forma dinámica, añadimos la dependencia de **Springdoc OpenAPI**:

```
<dependency>

        <groupId>org.springdoc</groupId>

        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

        <version>2.6.0</version>

</dependency>
```

Al levantar la aplicación, esta librería analiza tus clases controladores de forma reflexiva y expone una consola interactiva en la URL: http://localhost:8080/swagger-ui/index.html
Desde esta interfaz visual, cualquier desarrollador puede ver el catálogo completo de endpoints de tu sistema, inspeccionar la estructura de los JSON de entrada/salida y realizar peticiones de prueba en vivo (incluso pegando cabeceras de tokens JWT).

4. **Monitoreo en Producción: Spring Boot Actuator**
   Cuando una aplicación está corriendo en producción, los desarrolladores y el equipo de operaciones (DevOps) necesitan saber si el sistema está saludable. **Spring Boot Actuator** añade endpoints de inspección listos para usar agregando su starter:

xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Al configurarlo en el archivo de propiedades, expone endpoints críticos como:

- /actuator/health: Retorna un JSON indicando si la aplicación está arriba ("status": "UP") y si la base de datos responde correctamente.
- /actuator/metrics: Expone métricas detalladas de consumo de memoria RAM, uso de la CPU y cantidad de peticiones HTTP procesadas por segundo.

5. **Despliegue en la Nube mediante Dockerización**
   Para evitar el clásico problema de *"en mi máquina sí funciona pero en el servidor falla"*, empaquetamos la aplicación de Spring Boot en un contenedor aislado utilizando **Docker**.
   Primero, generamos el archivo comprimido .jar ejecutable de la aplicación ejecutando en la terminal de tu proyecto:

```
/mvnw clean package  # (O gradle build si usas Gradle)
```

   Posteriormente, creamos un archivo de texto plano llamado **Dockerfile** en la raíz del proyecto para definir cómo se construirá el contenedor:

```
dockerfile
# 1. Usar una imagen base ligera de Java (JRE de OpenJDK 17 o 21)
FROM eclipse-temurin:17-jre-alpine

# 2. Definir el directorio de trabajo dentro del contenedor
WORKDIR /app

# 3. Copiar el archivo .jar compilado desde nuestra máquina local hacia el contenedor
COPY target/tienda-0.0.1-SNAPSHOT.jar app.jar

# 4. Exponer el puerto por el que escucha Tomcat
EXPOSE 8080

# 5. Comando de arranque definitivo para ejecutar la aplicación al iniciar el contenedor
ENTRYPOINT ["java", "-jar", "app.jar"]
```

   Con este contenedor listo, servicios de alojamiento modernos en la nube (como **Railway, Render, AWS o Google Cloud**) pueden leer este archivo e implementar sistemas de integración continua (CI/CD) para actualizar la API automáticamente cada vez que se hace un git push.

Sección Especial: Autoevaluación

1. ***¿Por qué se recomienda la inyección de dependencias por constructor sobre la inyección por campos (@Autowired) en Spring?***

A) Permite crear objetos inmutables (final), facilita los tests unitarios sin levantar el contenedor de Spring y previene dependencias circulares ocultas.

B) Es significativamente más rápido en tiempo de ejecución porque evita el uso de la reflexión de Java.

C) Consume menos memoria RAM ya que no requiere la creación de proxies dinámicos en el contenedor.

D) Es la única forma permitida para inyectar beans opcionales en las nuevas versiones de Spring Boot 3.

2. ¿Cuál es el orden de ejecución correcto durante la inicialización de un Bean de Spring que contiene un constructor, una propiedad inyectada por @Autowired y un método anotado con @PostConstruct?

A) @PostConstruct -> Constructor -> Inyección @Autowired

B) Inyección @Autowired -> Constructor -> @PostConstruct

C) Constructor -> Inyección @Autowired -> @PostConstruct

D) Constructor -> @PostConstruct -> Inyección @Autowired

3. Al utilizar @RestController en Spring MVC, ¿cómo gestiona por defecto la conversión de un objeto Java complejo a una representación JSON en la respuesta HTTP?

A) Mediante la serialización nativa de objetos Java basada en ObjectOutputStream codificada en Base64.

B) Utilizando un HttpMessageConverter registrado, habitualmente MappingJackson2HttpMessageConverter si Jackson está en el class-path.

C) A través del motor de plantillas Thymeleaf configurado internamente en el DispatcherServlet.

D) Convirtiendo el objeto a un mapa de tipo HashMap mediante reflexión antes de enviarlo por el socket TCP.

4. ***¿Qué problema de rendimiento clásico ocurre frecuentemente al recuperar entidades con relaciones @OneToMany configuradas con carga perezosa (LAZY) si se intenta acceder a dichas colecciones fuera del contexto transaccional o se recorren ineficientemente?***

A) El error de recursión infinita por desbordamiento de pila (StackOverflowError).

B) Un bloqueo exclusivo de escritura pessimista en toda la tabla (Table Lock).

C) La saturación del pool de conexiones HikariCP por fugas de cursores abiertos.

D) El problema de consultas N+1, donde se ejecuta una consulta inicial para la entidad padre y N consultas adicionales para obtener los hijos de cada iteración.

5. ¿Cuál es el principal beneficio arquitectónico de utilizar @RestControllerAdvice junto con @ExceptionHandler en una aplicación Spring Boot?

A) Centralizar el manejo de errores de toda la aplicación para traducirlos de manera uniforme a respuestas HTTP con códigos de estado y cuerpos de error estandarizados.

B) Reiniciar automáticamente el contenedor de beans de Spring si ocurre un error crítico de tipo RuntimeException.

C) Forzar un rollback inmediato de todas las transacciones abiertas, incluso si la excepción es de tipo checked.

D) Desactivar los logs de traza de error en la consola para mejorar el rendimiento de producción.

6. ***En las versiones recientes de Spring Security (6.x / Spring Boot 3.x), ¿cómo se define y registra de forma recomendada la configuración de la cadena de filtros de seguridad (SecurityFilterChain)?***

A) Heredando de la clase abstracta WebSecurityConfigurerAdapter y sobreescribiendo el método configure (HttpSecurity).

B) Declarando un archivo XML de configuración de seguridad ubicado en el directorio META-INF/spring.

C) Exponiendo un Bean de tipo SecurityFilterChain dentro de una clase anotada con @Configuration utilizando expresiones lambda.

D) Anotando la clase principal con @DisableSpringSecurity y escribiendo filtros nativos de Servlet puros.

7. ¿Por qué al integrar autenticación basada en JWT es una práctica estándar implementar un filtro personalizado que extienda OncePerRequest-Filter en lugar de un filtro estándar de Servlet?

A) Porque es el único mecanismo capaz de descifrar la firma criptográfica HMAC-SHA256 de los tokens.

B) Para garantizar que la lógica de validación del token y la asignación del contexto de seguridad se ejecuten exactamente una vez por cada petición HTTP, evitando ejecuciones duplicadas en reenvíos o redirecciones internas.

C) Para almacenar obligatoriamente la sesión del usuario en la memoria caché del servidor HTTP subyacente.

D) Porque Spring Security prohibe explícitamente el uso de filtros estándar de tipo javax.servlet.Filter o jakarta.servlet.Filter.

8. ¿Cuál es la diferencia principal entre @WebMvcTest y @SpringBootTest al realizar pruebas automatizadas en Spring Boot?

A) @WebMvcTest carga el contexto completo de la aplicación incluyendo bases de datos reales, mientras que @SpringBootTest solo carga componentes vacíos.

B) No existe ninguna diferencia; ambos cargan de forma idéntica todos los beans de configuración y repositorios de datos.

C) @WebMvcTest solo permite pruebas con JUnit 4 y mockend sin MockMvc, requiriendo @SpringBootTest para utilizar peticiones simuladas HTTP.

D) @WebMvcTest carga un contexto reducido enfocado únicamente en la capa web y los controladores (mockeando por defecto los servicios), mientras que @SpringBootTest carga el contexto completo de la aplicación.

9. ¿Qué ocurre si inyectas un Bean con scope prototype dentro de un Bean con scope singleton utilizando la inyección estándar @Autowired?

A) El bean prototype se comporta de forma efectiva como un singleton, ya que la inyección ocurre una sola vez cuando se inicializa el bean contenedor singleton.

B) Spring lanza una excepción en tiempo de compilación impidiendo el despliegue del artefacto.

C) El contenedor crea una instancia nueva del bean prototype en cada invocación de un método del singleton.

D) Spring convierte automáticamente el bean contenedor singleton en un bean de tipo prototype.

10. ¿Por qué un método anotado con @Transactional puede fallar al no iniciar una transacción cuando es invocado directamente desde otro método dentro de la misma clase (self-invocation)?

A) Porque la anotación @Transactional solo funciona de forma exclusiva en interfaces y está prohibida en clases de implementación.

B) Porque el recolector de basura de la máquina virtual de Java elimina los metadatos de las anotaciones internas.

C) Porque las transacciones en Spring se gestionan mediante proxies dinámicos (Spring AOP); las llamadas internas dentro de la misma instancia omiten el proxy y ejecutan el método directamente en el objeto real.

D) Porque las bases de datos relacionales bloquean el hilo de ejecución principal ante auto-invocaciones concurrentes.

Respuestas y explicaciones

1. A) Permite crear objetos inmutables (final), facilita los tests unitarios sin levantar el contenedor de Spring y previene dependencias circulares ocultas.

- Explicación: Correcto! La inyección por constructor asegura que el objeto no se pueda construir en un estado inválido, facilita la inmutabilidad y permite instanciar la clase con new

  pasando mocks directamente en las pruebas unitarias.

2. C) Constructor -> Inyección @Autowired -> @PostConstruct

- Explicación: Correcto! El ciclo de vida estándar de Spring establece que primero se instancia la clase mediante el constructor, a continuación se realizan las inyecciones de dependencias necesarias, y finalmente se ejecuta el método de inicialización @PostConstruct.

3. B) Utilizando un HttpMessageConverter registrado, habitualmente MappingJackson2HttpMessageConverter si Jackson está en el classpath.

- Explicación: Correcto! Spring MVC utiliza la infraestructura de HttpMessageConverter y, por defecto, delega en Jackson para serializar objetos Java a JSON de forma transparente.

4. D) El problema de consultas N+1, donde se ejecuta una consulta inicial para la entidad padre y N consultas adicionales para obtener los hijos de cada iteración.

- Explicación: Correcto! El problema de N+1 ocurre cuando se accede a colecciones perezosas de manera iterativa sin un JOIN FETCH o un EntityGraph, degradando drásticamente el rendimiento de la base de datos.

5. A) Centralizar el manejo de errores de toda la aplicación para traducirlos de manera uniforme a respuestas HTTP con códigos de estado y cuerpos de error estandarizados.

- Explicación: Correcto! @RestControllerAdvice permite interceptar excepciones de forma transversal y global, eliminando la necesidad de repetir bloques try-catch en cada controlador individual.

6. C) Exponiendo un Bean de tipo SecurityFilterChain dentro de una clase anotada con @Configuration utilizando expresiones lambda.

- Explicación: Correcto! El enfoque moderno se basa en componentes y lambdas, exponiendo directamente un Bean de tipo SecurityFilterChain en el contexto de Spring.

7. B) Para garantizar que la lógica de validación del token y la asignación del contexto de seguridad se ejecuten exactamente una vez por cada petición HTTP, evitando ejecuciones duplicadas en reenvíos o redirecciones internas.

- Explicación: Correcto! OncePerRequestFilter garantiza una única ejecución por ciclo de solicitud, previniendo procesamiento redundante en contenedores de Servlet que realizan inclusiones o reenvíos internos.

8. D) @WebMvcTest carga un contexto reducido enfocado únicamente en la capa web y los controladores (mockeando por defecto los servicios), mientras que @SpringBootTest carga el contexto completo de la aplicación.

- Explicación: Correcto! @WebMvcTest es una prueba unitaria/de integración ligera ideal para controladores, optimizando el tiempo de ejecución al no levantar servicios, repositorios ni bases de datos.

9. A) El bean prototype se comporta de forma efectiva como un singleton, ya que la inyección ocurre una sola vez cuando se inicializa el bean contenedor singleton.

- Explicación: Correcto! Como el bean singleton se crea una sola vez al arrancar la aplicación, la dependencia prototype se resuelve en ese momento y nunca se vuelve a recrear en llamadas posteriores.

10. C) Porque las transacciones en Spring se gestionan mediante proxies dinámicos (Spring AOP); las llamadas internas dentro de la misma instancia omiten el proxy y ejecutan el método directamente en el objeto real.

- Explicación: Correcto! El proxy intercepta únicamente las llamadas externas que entran al bean. Una llamada interna (this.miMetodoTransaccional()) salta el proxy y por ende no activa la infraestructura transaccional.

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- https://docs.spring.io/spring-boot/
- https://spring.io/guides/gs/handling-form-submission
- https://dev.mysql.com/doc/refman/8.0/en/
- https://www.youtube.com/watch?v=DRQZkBhkpgY&list=PL0W7WdoXu6O8lxfzq8AusxVfR4uP0Tpmi&index=3
- https://www.youtube.com/watch?v=U-2p3FvnQcw&list=PLHC-DiueSVwLdYppl-07_lK0DYnK52qKe&index=6
- https://www.youtube.com/watch?v=pinK_MaUs-8&list=PLHC-DiueSVwLdYppl-07_lK0DYnK52qKe&index=7
- https://www.tutorialesprogramacionya.com/springbootya/